

1.

Experiment	Window	Step	Train/Val/Test Size	Train MSE	Val MSE	Test MSE
Baseline	10	15	227/57/31	72.61	188.21	280.56
1	10	5	680/171/94	5.22	8.32	15.87
2	10	2	1700/425/236	1.45	2.47	1.51
3	15	2	1699/425/235	1.69	2.91	7.18

I first reduced the step size from 15 to 2 to increase the density of training data. This led to a significant performance boost, with test MSE dropping from 280.56 to 1.51. Then, I increased the window size from 10 to 15, hoping to capture more temporal context. However, this change increased test MSE to 7.18. These results suggest that reducing step size effectively improves model performance by increasing sample size, while increasing the window size does not always help and may even lead to overfitting.

2.

Experiment	Window	Step	Train/Val/Test Size	Train MSE	Val MSE	Test MSE
Baseline	10	15	227/57/31	670.97	1268.05	1335.32
1	10	5	680/171/94	931.73	696.00	1105.51
2	10	2	1700/425/236	923.68	923.58	1001.36
3	15	2	1699/425/235	899.49	959.98	1029.22

- (i) Upon examining the data, I observed that the 'Volume' feature has a much larger numerical scale compared to the other features like 'Open', 'High', 'Low', and 'Close', which are all on a relatively similar and smaller scale. This mismatch likely caused the model to overemphasize 'Volume' after it was added, even though it may not be strongly correlated with the target variable. Without proper feature scaling, this imbalance can negatively affect the model's learning process, which aligns with the significant drop in performance observed after including 'Volume'.

Experiment	Feature	Train MSE	Val MSE	Test MSE
Baseline	Open, High, Low, Close	1.45	2.47	1.51
1	Open, High, Low	2.65	2.77	1.96
2	Open, High, Close	2.73	2.56	1.79
3	High, Low, Close	2.63	2.67	1.46
4	High, Low	1.27	2.77	2.61
5	High, Closs	1.20	2.99	1.59
6	Open, High	1.22	3.23	2.23
7	High	1.33	2.85	1.90

- (ii) I kept the window size at 10 and the step size at 2 throughout the experiments. Since the prediction target is the High price, I included High as an input feature in all combinations to help the model learn its temporal dynamics. I then experimented with adding different combinations of features such as 'Low', 'Close', and 'Open' alongside 'High' to evaluate their impact on model performance. The results showed that the combination of 'High', 'Low', and 'Close' achieved the best performance, with the lowest test MAE of 1.46. This indicates that these features provide the most useful complementary information for improving the accuracy of the model's predictions.

3.

Normalization	Train MSE	Val MSE	Test MSE
N	923.68	923.58	1001.36
Y	2.90	3.74	2.68

Throughout all experiments, the window size was consistently set to 10 and the step size to 2. When 'Volume' was introduced as an additional feature, the Test MSE increased drastically, suggesting that the model struggled with the new input due to its large numerical scale. To address this issue, standardization was applied to normalize the feature values. As a result, the Test MSE dropped significantly from 1001.36 to 2.68. This substantial improvement demonstrates the critical role of feature scaling in time series prediction tasks, especially when features with varying magnitudes are included in the dataset.

4. Setting the window size smaller than the step size reduces overlap between

training samples, which may help prevent data leakage and ensure more independent samples. However, this is not always ideal—larger windows often capture more temporal context. The choice depends on task goals and model type (Brownlee, 2017).

Reference:

Brownlee, J. (2017). *Introduction to Time Series Forecasting with Python*. Machine Learning Mastery.

5. One effective method for time-series data augmentation is **window slicing**, where overlapping or non-overlapping subsequences are extracted from the original series to create additional training samples. This approach maintains the temporal structure while increasing data diversity, especially useful when data is limited. It is simple to implement and commonly used in forecasting and classification tasks. Variants like sliding windows with random starting points can further enrich the dataset. However, care must be taken to avoid introducing highly similar samples that may lead to overfitting.

Reference:

Um, T. et al. (2017). *Data augmentation of wearable sensor data for Parkinson's disease monitoring using convolutional neural networks*. ICMI.

6. - **Convolution-based models:** Convolutional models use a fixed window size during inference. The window slides over the time series to process subsequences, making predictions for each. Overlapping windows can be used to capture more context, with padding added to maintain consistency at the sequence edges.
 - **Recurrent-based models:** Recurrent models, such as LSTMs or GRUs, process the sequence step-by-step and do not require a fixed window size. During inference, the model updates its hidden states sequentially. However, long sequences may require truncation or sliding windows for efficiency.
 - **Transformer-based models:** Transformers process the entire sequence in parallel, capturing long-range dependencies with their self-attention mechanism. For long sequences, sliding windows or memory compression techniques may be used to manage the input size during inference.